

Aging Apprentice Software Development Lifecycle

Standards and best practices

Development Environment Terminology

I refer to the my development setup as the **development environment** or **development workflow**. This encompasses all the tools and processes used for coding, version control, project management, and documentation.

- **Local Tools:**
 - Visual Studio Code,
 - Git
- **Remote Tools:**
 - GitHub (repositories, projects, wikis)

Guidelines:

This is how to set up a new project.

1. Create a Clear Repository Structure

- **Organize Files:** Use folders to categorize different types of files (e.g., `src` for source code, `docs` for documentation).
- **README.md:** Write a detailed README file to explain the purpose of your project, how to set it up, and any other relevant information.

2. Use Issues and Labels

- **Track Tasks:** Create issues for tasks, bugs, and features. This helps you keep track of what needs to be done.
- **Labeling:** Use labels to categorize issues (e.g., `bug`, `enhancement`, `question`). This makes it easier to prioritize and manage tasks.

3. Utilize Project Boards

- **Kanban Style:** Use GitHub's project boards to visualize your workflow. You can create columns like `To Do`, `In Progress`, and `Done` to track the status of tasks.
- **Flexibility:** This is great for uncommitted resources as you can move tasks around based on availability and priority.

4. Branching Strategy

- **Main Branch:** Keep your main branch stable and use feature branches for new developments.
- **Pull Requests:** Use pull requests to review and merge changes. This helps maintain code quality and allows you to track changes.

5. Documentation

- **Wiki:** Use GitHub's wiki feature to create detailed documentation for your project.
- **Code Comments:** Write comments in your code to explain complex logic and decisions.

6. Automation

- **GitHub Actions:** Set up GitHub Actions for automated testing, building, and deployment. This can save time and ensure consistency.

7. Community Engagement

- **Collaborators:** If you want others to contribute, add collaborators and encourage them to participate.
- **Discussions:** Use GitHub Discussions to engage with the community, ask questions, and share ideas.

8. Regular Updates

- **Commit Regularly:** Make regular commits to keep track of progress and changes.
- **Changelog:** Maintain a changelog to document major updates and changes.

By following these tips, you can effectively manage your hobby project on GitHub, even with uncommitted resources. Do you have any specific goals or features in mind for your project?

Development Workflow

● **Processes:** Branching strategy, committing changes, pull requests, continuous integration
Using Visual Studio Code with Git locally and GitHub remotely is a great setup! Here's a branching strategy and process that can help you manage your project effectively:

Branching Strategy

1. Main Branch (`main`)

- **Purpose:** This is your stable branch where the production-ready code resides.
- **Usage:** Only merge tested and reviewed code into this branch.

2. Development Branch (`dev`)

- **Purpose:** This branch is used for integrating features and bug fixes before they are ready for the main branch.
- **Usage:** Merge feature branches into this branch for testing and integration.

3. Feature Branches (`feature/feature-name`)

- **Purpose:** Each new feature or improvement gets its own branch.
- **Usage:** Create a new branch from `dev` for each feature. Once the feature is complete and tested, merge it back into `dev`.

4. Bugfix Branches (`bugfix/bug-name`)

- **Purpose:** Similar to feature branches, but specifically for fixing bugs.
- **Usage:** Create a new branch from `dev` for each bug fix. Merge it back into `dev` once the fix is complete.

5. Release Branches (`release/release-version`)

- Purpose:** Used for preparing a new production release.

- Usage:** Create a release branch from `dev` when you are ready to prepare a new release. Perform final testing and fixes here before merging into `main`.

6. Hotfix Branches (`hotfix/hotfix-name`)

- Purpose:** For urgent fixes that need to be applied directly to the `main` branch.

- Usage:** Create a hotfix branch from `main`, apply the fix, and merge it back into `main` and `dev`.

Process

1. Creating Branches

- Feature/Bugfix:** `git checkout -b feature/feature-name dev`

- Release:** `git checkout -b release/release-version dev`

- Hotfix:** `git checkout -b hotfix/hotfix-name main`

2. Committing Changes

- Regular Commits:** Make regular commits with clear messages to track progress.

- Commit Message Format:** Use a format like `type(scope) : description` (e.g., `feat(login) : add user authentication`).

3. Pull Requests

- Creating PRs:** Open pull requests on GitHub to merge feature/bugfix branches into `dev`, and `dev` into `main`.

- Review:** Review and discuss changes in the pull request before merging.

4. Merging

- Merge Strategy:** Use `git merge` or `git rebase` depending on your preference. `Rebase` keeps a linear history, while `merge` preserves the commit history.

- Conflict Resolution:** Resolve any merge conflicts during the pull request review process.

5. Continuous Integration

- Automated Tests:** Set up GitHub Actions to run automated tests on pull requests to ensure code quality.

- Builds:** Automate builds and deployments if applicable.

Example Workflow

1. **Start a Feature:** `git checkout -b feature/new-feature dev`

2. **Develop and Commit:** Make changes and commit regularly.

3. **Push to GitHub:** `git push origin feature/new-feature`

4. **Open PR:** Open a pull request to merge `feature/new-feature` into `dev`.

5. **Review and Merge:** Review the PR, resolve conflicts, and merge.

6. **Prepare Release:** `git checkout -b release/v1.0.0 dev`

7. **Final Testing:** Perform final testing and fixes.

8. **Merge Release:** Merge `release/v1.0.0` into `main`.

By following this strategy and process, you can maintain a clean and organized workflow for your project. Do you have any specific features or goals for your project that you'd like to discuss further?

Steps to Automate Testing

Absolutely! Automating testing when combining a feature branch with a release branch is a great way to ensure code quality and catch issues early. Here's how you can set it up using GitHub Actions:

1. Create a GitHub Actions Workflow

○**Location:** Place your workflow file in `.github/workflows/`.

○**File Name:** Name it something like `test-on-merge.yml`.

2. Define the Workflow

○**Trigger:** Set the workflow to trigger on pull requests targeting the release branch.

○**Jobs:** Define jobs to run your tests.

Example Generic Workflow File (`test-on-merge.yml`)

```
name: Test on Merge

on:
  pull_request:
    branches:
      - release/*

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Set up Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '14'

      - name: Install dependencies
        run: npm install

      - name: Run tests
        run: npm test
```

Explanation

●**Trigger:** The workflow triggers on pull requests to any branch that matches `release/*`.

●**Jobs:** The `test` job runs on the latest Ubuntu environment.

○**Checkout code:** Uses the `actions/checkout` action to check out the code.

○**Set up Node.js:** Sets up Node.js (adjust the version as needed).

○**Install dependencies:** Installs project dependencies.

○**Run tests:** Runs the tests using `npm test`.

Additional Tips

●**Custom Scripts:** Modify the workflow to fit your project's specific testing needs (e.g., Python, Java).

●**Notifications:** Set up notifications to alert you of test results.

●**Coverage Reports:** Integrate tools like Codecov to generate coverage reports.

By automating testing, you can ensure that your code is thoroughly checked before merging, reducing the risk of introducing bugs. Do you need help with setting up specific tests or integrating other tools into your workflow?

Example ESP32 Workflow File (`arduino-test.yml`)

Certainly! Here's an example of a GitHub Actions workflow file (`arduino-test.yml`) to test Arduino code on an ESP32:

```
name: Arduino ESP32 Test

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Set up Arduino CLI
        uses: arduino/setup-arduino-cli@v1
        with:
          version: 'latest'

      - name: Install ESP32 core
        run: arduino-cli core install esp32:esp32

      - name: Install libraries
        run: |
          arduino-cli lib install "Adafruit Unified Sensor"
          arduino-cli lib install "Adafruit BME280 Library"

      - name: Compile Sketch
        run: arduino-cli compile --fqbn esp32:esp32:esp32 your_sketch.ino

      - name: Upload Sketch (optional)
        run: arduino-cli upload -p /dev/ttyUSB0 --fqbn esp32:esp32:esp32 your_sketch.ino
```

```
if: github.event_name == 'push'
```

Explanation

- **Trigger:** The workflow triggers on pushes and pull requests to the `main` branch.
- **Jobs:** The `build` job runs on the latest Ubuntu environment.
 - **Checkout code:** Uses the `actions/checkout` action to check out the code.
 - **Set up Arduino CLI:** Sets up the Arduino CLI using the `arduino/setup-arduino-cli` action.
 - **Install ESP32 core:** Installs the ESP32 core using Arduino CLI.
 - **Install libraries:** Installs necessary libraries for your project.
 - **Compile Sketch:** Compiles the Arduino sketch for the ESP32 board.
 - **Upload Sketch:** Optionally uploads the sketch to the ESP32 board if the event is a push.

Additional Tips

- **Sketch File:** Replace `your_sketch.ino` with the name of your Arduino sketch file.
- **Serial Port:** Adjust the serial port (`/dev/ttyUSB0`) based on your setup.
- **Library Names:** Modify the library names to match the ones used in your project.

Example Python Workflow File (`python-test.yml`)

Certainly! Here's an example of a GitHub Actions workflow file (`python-test.yml`) to test Python code:

```
name: Python Test
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
  pull_request:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout code
```

```
        uses: actions/checkout@v2
```

```
      - name: Set up Python
```

```
        uses: actions/setup-python@v2
```

```
        with:
```

```
          python-version: '3.9'
```

```
      - name: Install dependencies
```

```
        run: |
```

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

```
- name: Run tests
run: |
  pytest
```

Explanation

- **Trigger:** The workflow triggers on pushes and pull requests to the `main` branch.
- **Jobs:** The `test` job runs on the latest Ubuntu environment.
 - **Checkout code:** Uses the `actions/checkout` action to check out the code.
 - **Set up Python:** Sets up Python using the `actions/setup-python` action.
 - **Install dependencies:** Installs project dependencies from the `requirements.txt` file.
 - **Run tests:** Runs tests using `pytest`.

Additional Tips

- **Python Version:** Adjust the Python version (3.9) as needed for your project.
- **Dependencies:** Ensure your `requirements.txt` file lists all necessary dependencies.
- **Test Command:** Modify the test command if you use a different testing framework (e.g., `unittest`, `nose`).

This workflow will help automate the testing of your Python code. Do you need any further customization or additional steps for your specific project?